

A tunnel built from two keys

WireGuard configured from nothing on two machines, brought up, pinged through, and then watched on the wire — where the ICMP has disappeared and only UDP remains. Captured from a running pair.

Command `qlab run vpn-lab`

VMs `vpn-lab-server 192.168.100.1 · vpn-lab-client 192.168.100.2`

Tunnel `10.10.0.1 ↔ 10.10.0.2, UDP 51820`

Outcome `qlab test vpn-lab` → 5 exercises, 28 checks, all passed

1. Two machines, and nothing configured

```
--- server ---
ens3                UP                10.0.2.15/24 metric 100 fec0::5054:ff:fe12:3456/64
fe80::5054:ff:fe12:3456/64
ens4                UP                192.168.100.1/24 fe80::5054:ff:fe00:101/64
--- client ---
ens3                UP                10.0.2.15/24 metric 100 fec0::5054:ff:fe12:3456/64
fe80::5054:ff:fe12:3456/64
ens4                UP                192.168.100.2/24 fe80::5054:ff:fe00:102/64
```

```
$ what is running when the lab boots
inactive
wg-quick@wg0: inactive
tunnel interfaces: 0

$ but the tools are there
wireguard-tools v1.0.20210914 - https://git.zx2c4.com/wireguard-tools/
OpenVPN 2.5.11 x86_64-pc-linux-gnu [SSL (OpenSSL)] [LZO] [LZ4] [EPOLL] [PKCS11] [MH/PKTINFO] [AEAD]
built on Jul  8 2026
```

The lab ships the tools and no configuration at all. That is deliberate: the exercise is the configuring, and there is very little of it. Everything in the next section was done from this starting point.

2. Keys first, and only the public halves travel

```
server public key : yx26lNwuiYiU6vrKLY5WEKyYocqityyJMI0w38DPFWk=
client public key : LZa8WmUaaTkASdU4vrT0TipmUd/1JhpUP+Z5AI5f9kg=

Only the public halves are exchanged. Neither private key leaves its machine.

$ the server config that produced (private key redacted)
[Interface]
Address = 10.10.0.1/24
ListenPort = 51820
PrivateKey = <redacted>

[Peer]
PublicKey = LZa8WmUaaTkASdU4vrT0TipmUd/1JhpUP+Z5AI5f9kg=
AllowedIPs = 10.10.0.2/32

$ wg-quick up wg0, on both
server: [#] ip link add wg0 type wireguard
server: [#] wg setconf wg0 /dev/fd/63
server: [#] ip -4 address add 10.10.0.1/24 dev wg0
server: [#] ip link set mtu 1420 up dev wg0
client: [#] ip link add wg0 type wireguard
client: [#] wg setconf wg0 /dev/fd/63
client: [#] ip -4 address add 10.10.0.2/24 dev wg0
client: [#] ip link set mtu 1420 up dev wg0
```

This is the whole of WireGuard's identity model. Each machine generates a keypair and **keeps the private half**; what gets exchanged is the public key, which is safe to print in a document like this one.

There are no certificates, no certificate authority, no handshake negotiation about who you are. A peer is a public key with a list of addresses it is allowed to use, and the config file is short enough to read in full.

Three fields carry the meaning:

- **Address** — the machine's address *inside* the tunnel. It has nothing to do with the LAN addresses in section 1.
- **Endpoint** — where to send the encrypted UDP. Only the client needs one: the server learns the client's endpoint from the first packet that arrives, which is why WireGuard copes with clients that move networks.
- **AllowedIPs** — a double-duty field, and the one that confuses people. On outgoing traffic it is a *route*: send these addresses to this peer. On incoming traffic it is an *access check*: a packet decrypted from this peer claiming a source outside its AllowedIPs is dropped. Routing and authorisation in one line.

`wg-quick up` then does by hand what you would otherwise type: create the interface, load the config, add the address, set an MTU of 1420 — 1500 minus the encapsulation overhead.

3. The tunnel, up

```
$ ip addr show wg0      (server)
wg0                     UNKNOWN      10.10.0.1/24
10.10.0.0/24 dev wg0 proto kernel scope link src 10.10.0.1

$ ping 10.10.0.1 from the client, through the tunnel
--- 10.10.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2025ms
rtt min/avg/max/mdev = 0.311/0.467/0.617/0.125 ms

$ wg show on the server  (keys shortened)
interface: wg0
  public key: yx26lNwuiYiU6vrKlY5W...
  private key: (hidden)
  listening port: 51820

peer: LZa8WmUaaTkASdU4vrT0...
  endpoint: 192.168.100.2:36396
  allowed ips: 10.10.0.2/32
  latest handshake: 3 seconds ago
  transfer: 564 B received, 476 B sent
```

Three packets, no loss, under a millisecond. And `wg show` reports what matters: a **latest handshake** a few seconds ago, and a byte count in both directions.

Note the endpoint the server recorded: `192.168.100.2:36396`. The client never declared a port — the server learned it from the packets that arrived. There is also no "connected" state to report, because WireGuard has no connection: it is UDP, it is stateless between handshakes, and `latest handshake` is the closest thing to a session.

4. What the network sees

```
tcpdump on the lab LAN while the client pings 10.10.0.1 through the tunnel.  
12:59:56.448653 IP 192.168.100.2.36396 > 192.168.100.1.51820: UDP, length 128  
12:59:56.448759 IP 192.168.100.1.51820 > 192.168.100.2.36396: UDP, length 128  
12:59:56.448894 IP 192.168.100.1.51820 > 192.168.100.2.36396: UDP, length 128  
12:59:57.466532 IP 192.168.100.2.36396 > 192.168.100.1.51820: UDP, length 128  
12:59:57.466597 IP 192.168.100.1.51820 > 192.168.100.2.36396: UDP, length 128  
12:59:57.466699 IP 192.168.100.1.51820 > 192.168.100.2.36396: UDP, length 128  
12:59:58.490520 IP 192.168.100.2.36396 > 192.168.100.1.51820: UDP, length 128  
12:59:58.490583 IP 192.168.100.1.51820 > 192.168.100.2.36396: UDP, length 128
```

Every packet is UDP to port 51820. The ICMP is inside, and not visible here.

The capture filter asked for `udp port 51820 or icmp`, on the interface the ping actually crosses. Every single line is UDP, all the same length. **No ICMP appears at all** — not because it was filtered out, but because on this wire it does not exist. The pings are inside the encrypted payload.

That is the thing worth seeing rather than being told: an observer on the LAN learns that two addresses are exchanging UDP on a known port, and how much, and when. It learns nothing about what — not the protocol, not the inner addresses, not the content.

Tunnel removed; the lab is back to the state the guide starts from.

5. And OpenVPN, for contrast

The lab installs OpenVPN alongside, and exercise 2 sets it up with a static key. The comparison is the point of having both: OpenVPN runs over TLS with negotiable ciphers, needs a certificate authority for anything beyond a shared static key, and its configuration runs to dozens of lines. WireGuard has one cipher suite and no negotiation, which is why there is nothing to choose and nothing to get wrong — and also why it cannot be adapted when that suite ages.

6. Verification

```
Exercise 1: All 9 checks passed  
Exercise 2: All 5 checks passed  
Exercise 3: All 3 checks passed  
Exercise 4: All 5 checks passed  
Exercise 5: All 6 checks passed  
Exercises run: 5  
Exercises passed: 5  
Exercises failed: 0  
Exercises skipped: 0  
All exercises passed!
```

7. What to take away

- A peer is a public key. Private keys never leave the machine that made them.

- `Address` is inside the tunnel; `Endpoint` is outside it. Keeping the two straight solves most of the confusion.
- `AllowedIPs` is both the route out and the filter in.
- The server does not need to know where the client is; it learns the endpoint from the traffic.
- There is no connection. `latest handshake` is how you tell a tunnel is alive.
- On the wire there is only UDP. What travels inside is not visible from outside.

`guide.md` in the plugin carries the exercises: WireGuard by hand, OpenVPN with a static key, traffic analysis with tcpdump, and the firewall rules around them.